RNN Foundations

Networks with memory - and why the memory fades

Parts of this chapter adapt LMU I2DL (slds-lmu), CC BY 4.0.

The reviews arrive

The [50] classifier grades each pomegranate; the boxes ship worldwide. Now the **reviews** arrive: "Fresh and sweet!" (3 words); an 11-word one naming Yerevan; a 42-word complaint; some in Armenian. Ops wants them read, translated, answered.

Zoom out: **every** headline AI system of the 2020s - ChatGPT included - is a **sequence model**. This chapter and the next are the road there.

Predict: feed a review into an MLP ([41_2]) - what breaks before you even train?

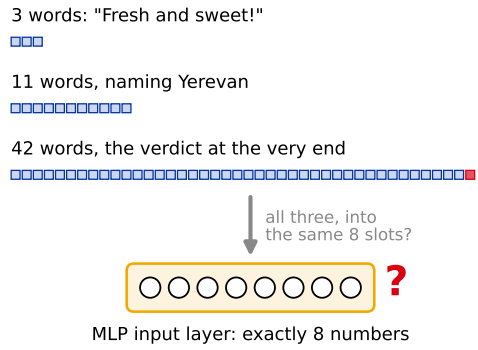
The reviews arrive

The [50] classifier grades each pomegranate; the boxes ship worldwide. Now the **reviews** arrive: "Fresh and sweet!" (3 words); an 11-word one naming Yerevan; a 42-word complaint; some in Armenian. Ops wants them read, translated, answered.

Zoom out: **every** headline AI system of the 2020s - ChatGPT included - is a **sequence model**. This chapter and the next are the road there.

Predict: feed a review into an MLP ([41_2]) - what breaks before you even train?

The input layer has a **fixed size**. Reviews do not.



Outline

Text refuses to fit

The recurrence idea

Training - and the fatal flaw

Text refuses to fit

Order is information, and length is a variable.

The sequence zoo

Text leads this chapter. Two close cousins share the same shape of problem: **speech** (a voice is a long list of air-pressure numbers) and **music** (a melody is a list of notes).

What they share: **order matters**, **length varies**, and dependencies can be **long-range** - the word that resolves a sentence's meaning may be 40 tokens back.

Text

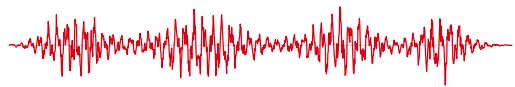
3 to 3,000 words

The | pomegranates | arrived | fresh | and | sweet

order matters: the verdict can sit in the last word

Speech

0.5 s to an hour



about 16,000 numbers per second

Music

8 to 8,000 notes



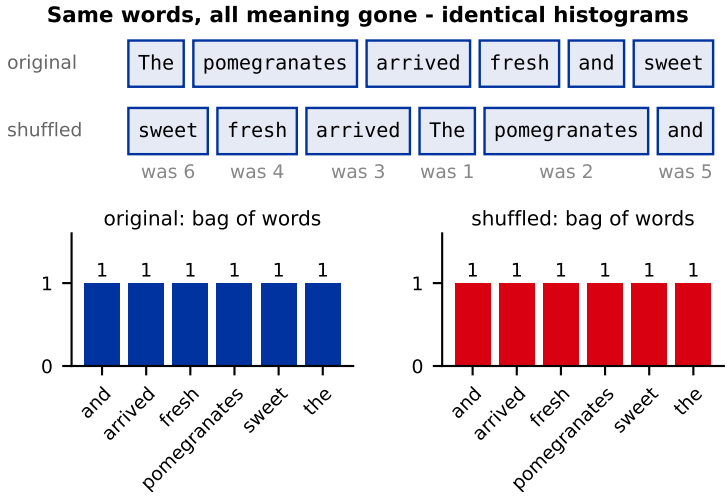
a note means something only next to its neighbours

The shuffle test

Take the review sentence and shuffle its words with one fixed permutation. Every word is identical; all meaning is gone.

The bag-of-words histograms are **pixel-identical**. Any order-blind model literally cannot tell the two apart.

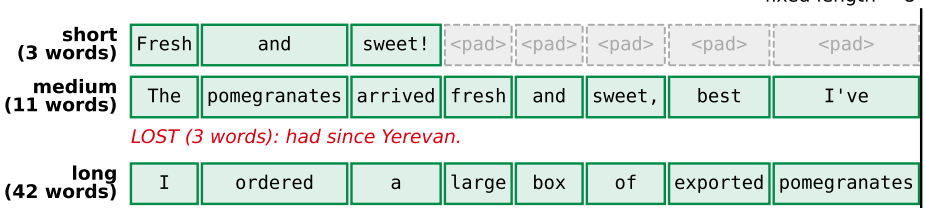
Callback: [48] did this to an image's *pixels*. Same demo, new axis.



The shoehorn hacks

How people force variable-length text into a fixed net - and the price of each:

Pad + truncate to a fixed length: waste on the short one, amputation on the long one



(2) Bag-of-words averaging fails the shuffle test by construction - it cannot see order at all. **(3) A separate weight per position** means "I went to Munich in 2009" and "In 2009, I went to Munich" must each relearn the same fact at a different position.

What about a fixed window?

You have already met two models that read a sequence through a **fixed window** of the last k tokens:

- the **1D convolution** from [48] - slide the same kernel along a sentence instead of a signal, and it reads a window of k tokens;
- the ch11 **name inventor** - an MLP that predicts the next letter from the last k letters (Bengio et al., 2003).

Both learn **local** patterns well - "-yan" closes an Armenian surname. Both are **blind beyond** k:

The **pomegranates** that my grandmother sent us from Goris last autumn **were** sweet.

A 4-word window in front of "were" sees only *from Goris last autumn*. It cannot tell whether one pomegranate or many was sent - the word that decides it is 10 words back.

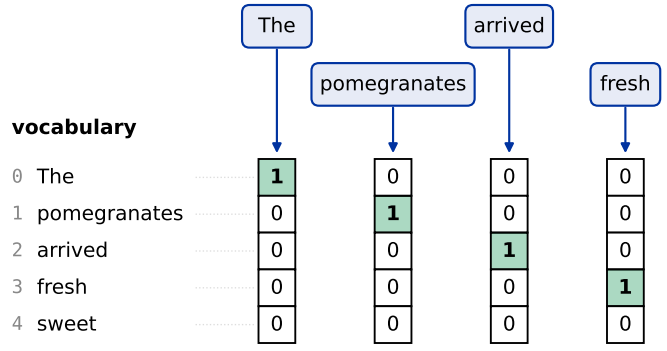
Stacking conv layers widens the window ([49]'s receptive field), but the width is fixed by the architecture before any text arrives. We want a memory that grows *with* the text: read one token at a time and carry a summary forward.

Tokens, minimally

To feed text to *any* network, words must first become vectors. For now, the simplest scheme: build a **vocabulary**, then represent each word as a **one-hot** vector.

> Callback: [03] one-hot encoded categorical features. Same trick, new victim - a *word* instead of a category.

The full tokenization story (subwords, vocabulary explosion, the Armenian fact) comes in the LLM chapter.

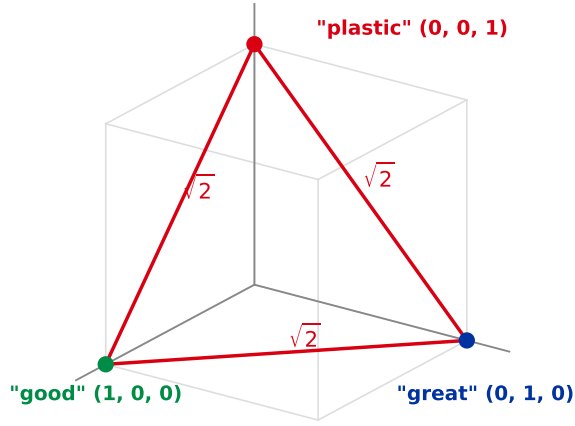


One-hot and its poverty

One-hot is all this chapter needs. It has one flaw worth seeing now:

In a 10,000-dimensional one-hot space, **every pair of words is equally far apart**. "good" is as far from "great" as it is from "plastic" - the geometry carries zero meaning.

With 3 words it fits in a picture: the words are the 3 corners of a cube next to the origin, and every edge between them is $\sqrt{2}$ long. Learned, dense word vectors fix this. They arrive with the attention chapter.



The recurrence idea

Read like a human: one word at a time, with memory.

You have already run a recurrence

[29]'s **exponential smoothing** forecast tomorrow from a running summary of the whole past:

$$\hat{y}_{t+1} = \alpha\, y_t + (1-\alpha)\, \hat{y}_t$$

New summary = a little of today + most of the old summary. Unroll it and each older reading's weight shrinks by the same factor every step: with $\alpha = 0.2$ the weights are $0.20, 0.16, 0.13, 0.10, \ldots$ - each one 0.8 of the one before.

	smoothing, [29]	**RNN, today**
summary	one number $\hat{y}_t$	a vector $\mathbf{z}^{[t]}$
input	one reading y_t	a word's vector $\mathbf{x}^{[t]}$
what to keep	fixed: $1-\alpha$	learned: a matrix $\mathbf{V}$
nonlinearity	none	tanh

An RNN is exponential smoothing that **learns what to keep**. Hold on to that 0.8: it comes back at the end of this lecture.

One new arrow

How a human reads: word by word, retaining a running summary - the **state**.

From [41_1]'s hidden layer $\mathbf{z} = \sigma(\mathbf{W}^\top \mathbf{x} + \mathbf{b})$ to

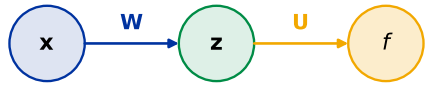
$$\mathbf{z}^{[t]} = \sigma\left(\mathbf{V}^\top \mathbf{z}^{[t-1]} + \mathbf{W}^\top \mathbf{x}^{[t]} + \mathbf{b}\right) :$$

one new term - the **hidden-to-hidden weights V** - and the result is a **recurrent neural network** (**RNN**).

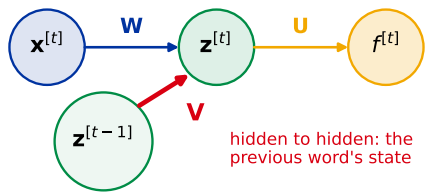
$\mathbf{z}^{[t]}$ is a function of the current input *and* the previous state - by recurrence, a summary of **everything read so far**.

History: Elman (1990), "Finding Structure in Time", after Jordan (1986) - the idea predates the web.

a hidden layer, [41_1]

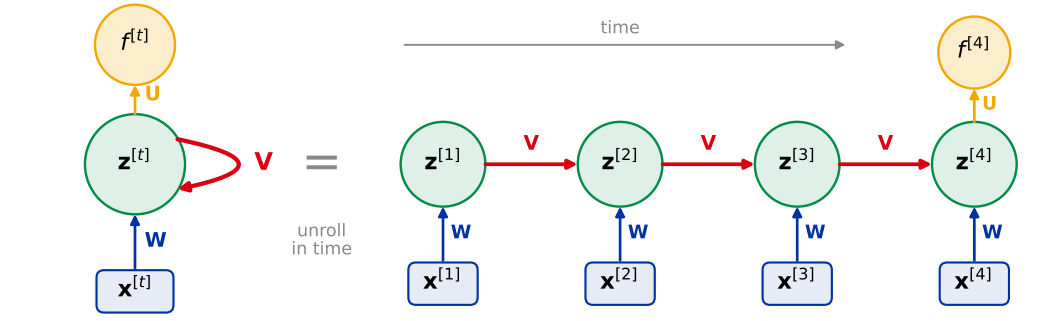


the same layer + one arrow = an RNN



Unrolling: one loop, drawn out in time

The red loop says "feed the state back in at the next word". Draw a separate copy of the cell for every word and the loop becomes a chain:



folded: one cell, a loop **unrolled: one copy per word, the same W, V, U in every copy**

The copies are not new parameters. A 4-word review and a 40-word review run the same cell, so the chain grows with the text while the network does not - that is how an RNN fixes the fixed-input problem. For seq-to-one sentiment only the last copy's f is used.

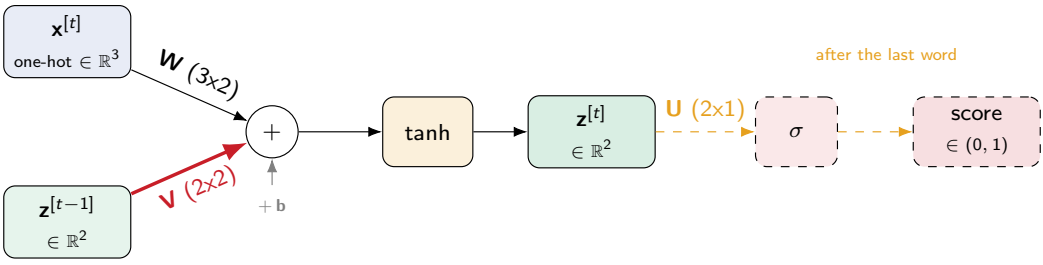
Anatomy of one RNN step

Same recurrence, now with concrete numbers to hold onto: a toy vocabulary of 3 words, a 2-d hidden state. Every arrow below carries its shape, so you always know exactly what multiplies what. **Predict:** the one-hot vector $\mathbf{x}^{[t]}$ meets $\mathbf{W}$ - what does that product actually pick out?

Anatomy of one RNN step

Same recurrence, now with concrete numbers to hold onto: a toy vocabulary of 3 words, a 2-d hidden state. Every arrow below carries its shape, so you always know exactly what multiplies what. **Predict:** the one-hot vector $\mathbf{x}^{[t]}$ meets $\mathbf{W}$ - what does that product actually pick out?

It selects ONE column of $\mathbf{W}^\top$ - equivalently, one row of $\mathbf{W}$. That row/column IS this word's own vector in $\mathbb{R}^2$ - the attention chapter calls it an **embedding**.

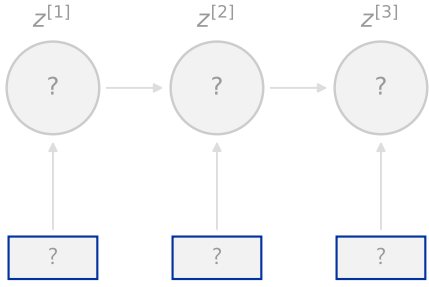


A complete forward pass, in real numbers

The chapter's Armenian line, first three words, through exactly the net above. **(Click to advance.)**

Toy net ready: 3-word vocabulary, 2-d state, $z^{[0]} = [0, 0]$

step 1 of 7



W (rows = words)

Ես	[+0.50, -0.50]
այս	[+1.00, +0.00]
ամենին	[+0.00, +1.00]

```
V = [[+0.5,-0.5],[+1.0,+0.0]]
b = [0,0]
```

A complete forward pass, in real numbers

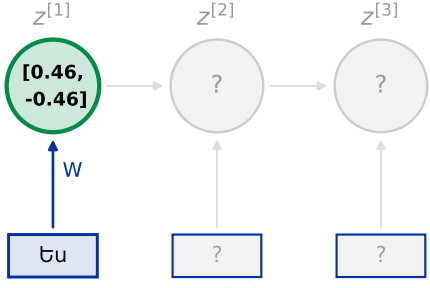
The chapter's Armenian line, first three words, through exactly the net above. **(Click to advance.)**

step 2 of 7

x[1] = one-hot(Ես) -> W^T x = row of W = [0.50, -0.50]

+ V^T z[0] = [0.00, 0.00] (state starts at zero) + b = [0, 0]

pre = [0.50, -0.50] -> tanh -> z[1] = [0.46, -0.46]



W (rows = words)

Ես	[+0.50, -0.50]
այս	[+1.00, +0.00]
ամենին	[+0.00, +1.00]

V = [[+0.5,-0.5],[+1.0,+0.0]]

b = [0,0]

A complete forward pass, in real numbers

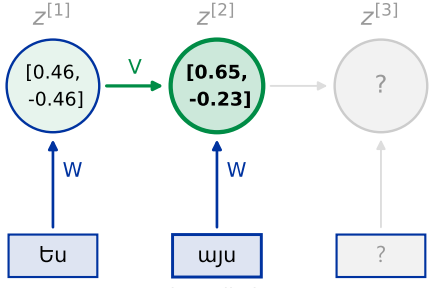
The chapter's Armenian line, first three words, through exactly the net above. **(Click to advance.)**

step 3 of 7

x[2] = one-hot(այս) -> W^T x = row of W = [1.00, 0.00]

+ V^T z[1] = [-0.23, -0.23] + b = [0, 0]

pre = [0.77, -0.23] -> tanh -> z[2] = [0.65, -0.23]



tanh applied at every state

W (rows = words)

Ես	[+0.50, -0.50]
այս	[+1.00, +0.00]
ամենին	[+0.00, +1.00]

V = [[+0.5,-0.5],[+1.0,+0.0]]

b = [0,0]

A complete forward pass, in real numbers

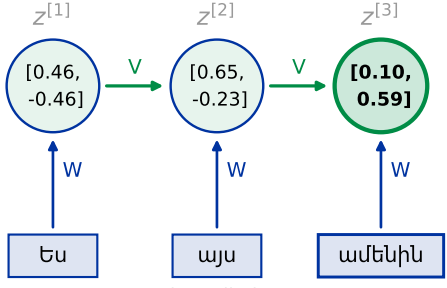
The chapter's Armenian line, first three words, through exactly the net above. **(Click to advance.)**

step 4 of 7

x[3] = one-hot(ամենին) -> W^T x = row of W = [0.00, 1.00]

+ V^T z[2] = [0.10, -0.32] + b = [0, 0]

pre = [0.10, 0.68] -> tanh -> z[3] = [0.10, 0.59]



W (rows = words)

Ես [+0.50, -0.50]

այս [+1.00, +0.00]

ամենին [+0.00, +1.00]

V = [[+0.5,-0.5],[+1.0,+0.0]]

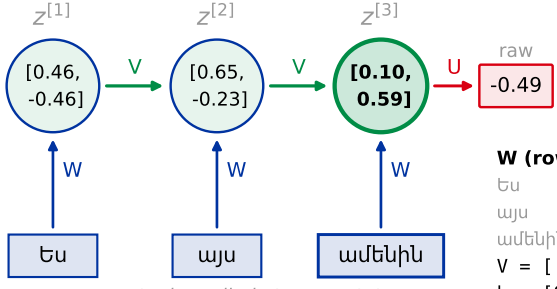
b = [0,0]

A complete forward pass, in real numbers

The chapter's Armenian line, first three words, through exactly the net above. **(Click to advance.)**

After the last word: readout $U^\top z^{[3]} + c$

(it happens once, not at every step)



W (rows = words)

Ես	[+0.50, -0.50]
այս	[+1.00, +0.00]
ամենին	[+0.00, +1.00]

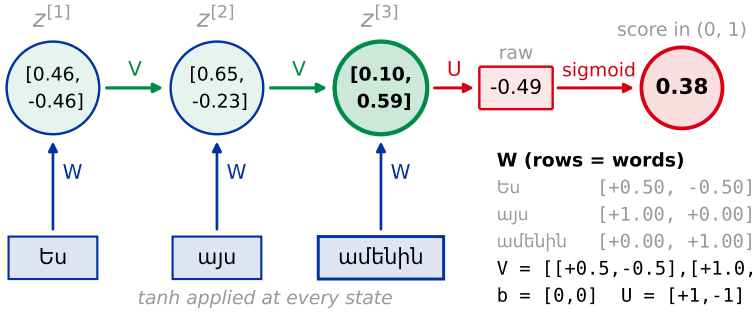
`V = [[+0.5,-0.5],[+1.0,+0.0]]`

`b = [0,0]  U = [+1,-1]  c = 0`

A complete forward pass, in real numbers

The chapter's Armenian line, first three words, through exactly the net above. **(Click to advance.)**

sigmoid(-0.49) = 0.38: one pass, one score

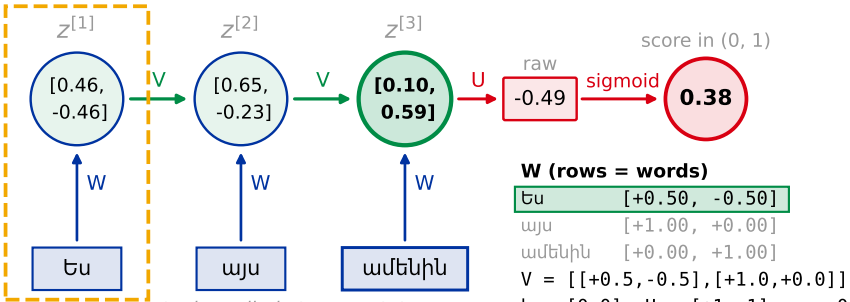


A complete forward pass, in real numbers

The chapter's Armenian line, first three words, through exactly the net above. **(Click to advance.)**

one-hot x W selects ONE row of W: that row IS Ես's own vector step 7 of 7

The attention chapter gives this row a name: an embedding.



Forward pass, summarized

All values rounded to 2 decimals; $\mathbf{z}^{[0]} = [0, 0]$, nothing read yet:

$$\mathbf{z}^{[1]} = \tanh(\mathbf{W}^\top \mathbf{x}_1 + \mathbf{b}) = [0.46, -0.46]$$
$$\mathbf{z}^{[2]} = \tanh(\mathbf{V}^\top \mathbf{z}^{[1]} + \mathbf{W}^\top \mathbf{x}_2 + \mathbf{b}) = [0.65, -0.23]$$
$$\mathbf{z}^{[3]} = \tanh(\mathbf{V}^\top \mathbf{z}^{[2]} + \mathbf{W}^\top \mathbf{x}_3 + \mathbf{b}) = [0.10, 0.59]$$
$$\text{score} = \sigma(\mathbf{U}^\top \mathbf{z}^{[3]}) = \sigma(-0.49) = 0.38$$

Three words, three applications of the **same** **W** and **V**, one final readout through **U**. Nothing hidden - every number above came straight out of the animation.

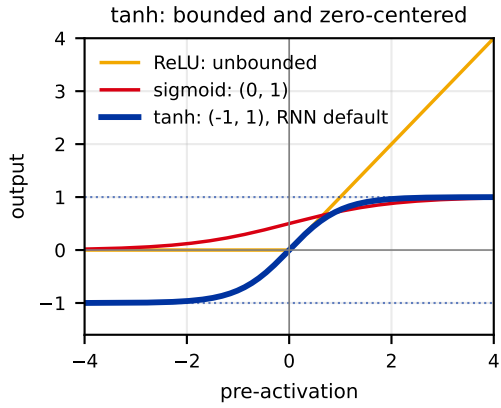
The weights were set by hand, not trained: 0.38 is a number to follow, not a verdict. In practice d_h = 64-512, not 2 - but the recipe is identical. And it is **fixed**: 3 words or 3,000, the whole past must fit into those d_h numbers - a lossy summary [54] will come back to.

Picking the activation: why tanh

Three candidates for the state's activation - only one is the RNN default:

- **tanh** (default): bounded in $(-1, 1)$ - the state literally **cannot blow up** across steps (the state - its gradient still can, as the training section shows); zero-centered, so both signs flow.
- **sigmoid**: also bounded, but $|\sigma'| \leq 1/4$ everywhere - one more shrinking factor on top of $\mathbf{V}$ every step.
- **ReLU**: unbounded above - feed that back through $\mathbf{V}$ every step and the state can genuinely **explode**, not just saturate.

Dead ReLU, one line: a unit whose pre-activation is always ≤ 0 outputs zero forever, and its gradient too - it never updates again.

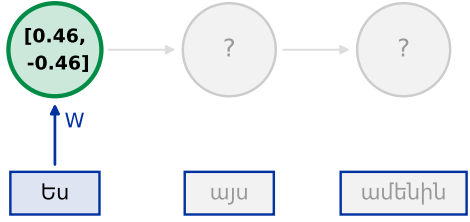


Same words, different order

The **same** toy net as the forward pass above, same 3 Armenian words - fed forward, then fed in the *opposite* order. **(Click to advance.)**

Forward: Ես -> այս -> ամենին

z[1] = tanh(W^T x) = [0.46, -0.46]



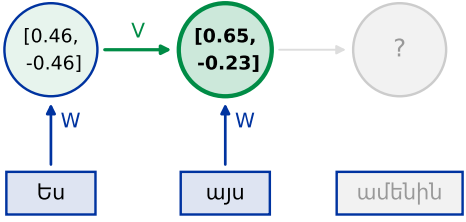
step 1 of 6

Same words, different order

The **same** toy net as the forward pass above, same 3 Armenian words - fed forward, then fed in the *opposite* order. **(Click to advance.)**

Forward: Ես -> այս -> ամենին

z[2] = tanh(V^T z[1] + W^T x + b) = [0.65, -0.23]



step 2 of 6

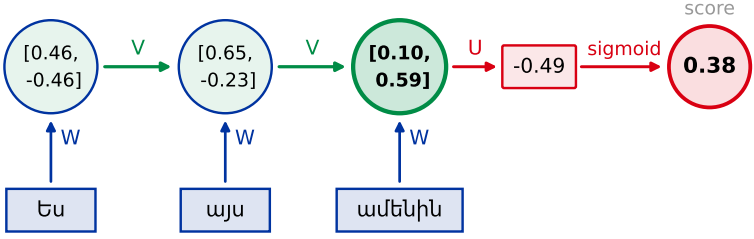
Same words, different order

The **same** toy net as the forward pass above, same 3 Armenian words - fed forward, then fed in the *opposite* order. **(Click to advance.)**

Forward: Ես -> այս -> ամենին

z[3] = tanh(V^T z[2] + W^T x + b) = [0.10, 0.59]

forward score = 0.38



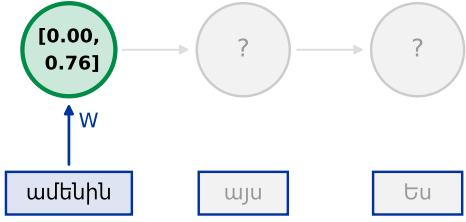
step 3 of 6

Same words, different order

The **same** toy net as the forward pass above, same 3 Armenian words - fed forward, then fed in the *opposite* order. **(Click to advance.)**

Reversed: ամենին -> այս -> Ես

z[1]' = tanh(W^T x) = [0.00, 0.76]



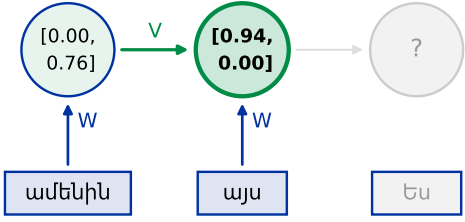
step 4 of 6

Same words, different order

The **same** toy net as the forward pass above, same 3 Armenian words - fed forward, then fed in the *opposite* order. **(Click to advance.)**

Reversed: ամենին -> այս -> Ես

z[2]' = tanh(V^T z[1]' + W^T x + b) = [0.94, 0.00]



step 5 of 6

Same words, different order

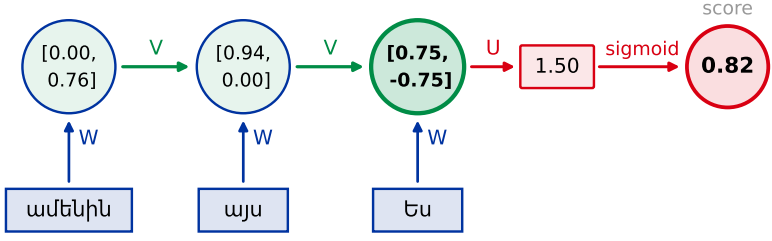
The **same** toy net as the forward pass above, same 3 Armenian words - fed forward, then fed in the *opposite* order. **(Click to advance.)**

Reversed: ամենին -> այս -> Ես

z[3]' = tanh(V^T z[2]' + W^T x + b) = [0.75, -0.75]

reversed score = 0.82

Same 3 words, different order: 0.38 vs 0.82



step 6 of 6

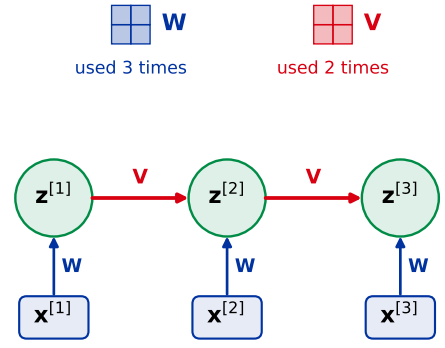
Parameter sharing

The *same* **W** and **V** at **every** step.
Parameter count is independent of sequence length: with $d_x = 1$, $d_h = 16$,

$$\underbrace{16}_{W} + \underbrace{256}_{V} + \underbrace{16}_{b} + \underbrace{16}_{U} + \underbrace{1}_{c} = \mathbf{305}$$

parameters - whether the review is 3 words or 40.

Convolution shares across **space** ([48]); recurrence shares across **time**. Same trick, new axis - and it kills hack (3): a pattern learned at position 2 works at position 17 for free.



one copy of each matrix: every blue arrow is the same W, every red arrow the same V

Predict-first: are pads harmless?

An RNN reads any length one review at a time. But training runs on **batches**, and a batch is a rectangle: every review in it needs the same length. The usual move is to fill the short ones with a <pad> token whose input vector is all zeros.

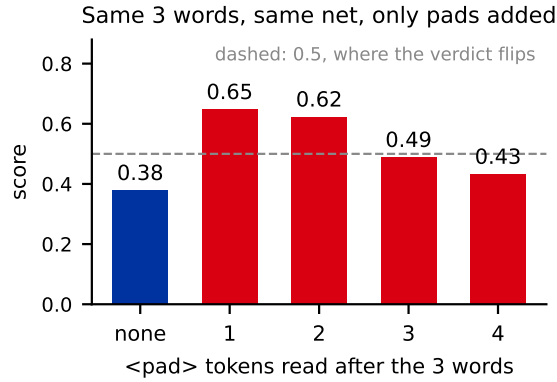
Predict: our toy net read the 3 Armenian words and scored 0.38. Feed it one zero <pad> after them. Does the score stay 0.38?

Predict-first: are pads harmless?

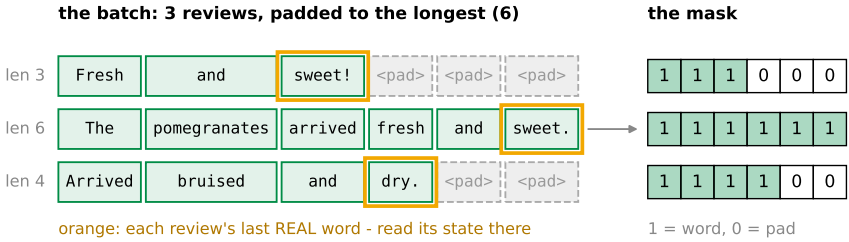
An RNN reads any length one review at a time. But training runs on **batches**, and a batch is a rectangle: every review in it needs the same length. The usual move is to fill the short ones with a <pad> token whose input vector is all zeros.

Predict: our toy net read the 3 Armenian words and scored 0.38. Feed it one zero <pad> after them. Does the score stay 0.38?

No: **0.65**, and the verdict flips. A zero input is not "nothing": the state still goes through $\mathbf{V}$ and tanh, $\mathbf{z}^{[4]} = \tanh(\mathbf{V}^\top \mathbf{z}^{[3]} + \mathbf{0} + \mathbf{b}) = [0.56, -0.05]$. Every pad is one more step of the recurrence.



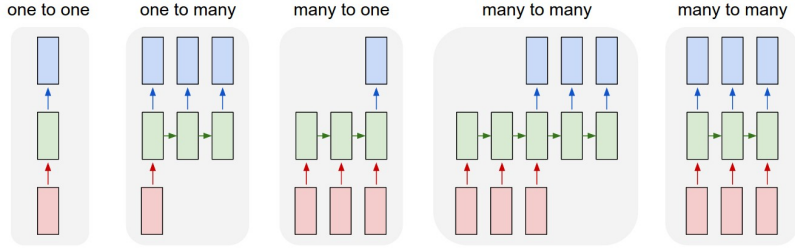
The fix: pad, then mask



- where the 0/1 **mask** is 0, the state is copied ($\mathbf{z}^{[t]} = \mathbf{z}^{[t-1]}$): as if the pad was never read (zeroing it would erase the review);
- each review is read at **its own** last real word, not at column 6;
- in next-word prediction, the loss skips the pad positions.

PyTorch: `pad_sequence` pads; `pack_padded_sequence(x, lengths)` stops each review at its own length. Hugging Face tokenizers return the mask as `attention_mask` (LLM chapter).

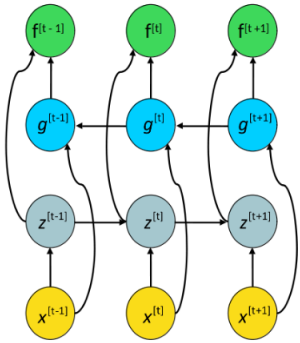
The taxonomy



- **seq-to-one**: sentiment (our reviews). One readout, at the end: $\sigma(\mathbf{U}^\top \mathbf{z}^{[T]})$.
- **one-to-seq**: image captioning.
- **seq-to-seq, delayed** (4th): translation. Read everything first; then a second RNN writes one word per step, each fed back in ([54]).
- **seq-to-seq, synced** (5th): language modeling, tagging. The *same* $\mathbf{U}$ reads every state, $\hat{\mathbf{y}}^{[t]} = \text{softmax}(\mathbf{U}^\top \mathbf{z}^{[t]})$. $\mathbf{z}^{[t]}$ has seen $\mathbf{x}^{[1]}, \ldots, \mathbf{x}^{[t]}$, so $\hat{\mathbf{y}}^{[1]}$ knows only $\mathbf{x}^{[1]}$. Language model: $\hat{\mathbf{y}}^{[t]}$ guesses $\mathbf{x}^{[t+1]}$.

Variants: bidirectional (next slide), stacked. "many-to-X" = our "seq-to-X". Diagram: Karpathy (2015), "The Unreasonable Effectiveness of RNNs".

Reading both ways: bidirectional RNNs



The **forward** states $z^{[t]}$ run left→right; the **backward** states $g^{[t]}$ run right→left; each output $f^{[t]}$ reads *both*.

A plain RNN only sees the **past**. But the right reading of a word often depends on what comes *after*:

- "**Teddy** *Roosevelt*" vs "**Teddy** *bears*" - is "Teddy" a name? You only know from the *next* word.

A **bidirectional** RNN (Schuster & Paliwal, 1997) runs one RNN each way and concatenates their states, so **every position sees the whole sequence**.

Great for **encoding** a sequence you already have (tagging, sentiment, the encoder half of translation). The catch: it needs the *whole input up front*, so it cannot **generate** or stream token by token - the setting [54] cares about.

Training - and the fatal flaw

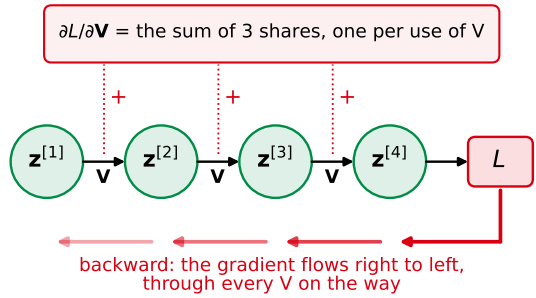
Unroll it, and it is just a deep network.

Backprop through time

The unrolled RNN is a deep feed-forward net with **tied weights**: [42]'s `loss.backward()` handles it unchanged. A tied weight collects the **sum** of its gradients from every step, like a shared conv kernel ([48]):

$$\frac{\partial L}{\partial \mathbf{V}} = \sum_{t=2}^{T} \frac{\partial L}{\partial \mathbf{V}}\Big|_{\text{use at step } t}$$

For seq-to-one sentiment, L is the log-loss of the final score against the review's label, and it sits only at the **final** step - so its gradient must travel back through every recurrent connection.



Very long texts: backprop through only the last k steps (**truncated** BPTT). Formalized as "backpropagation through time" by Werbos (1990) - four years after backprop itself (Rumelhart, Hinton & Williams, 1986).

Predict-first: the fading number

Strip the net down to 1-d: a scalar state $h^{[t]} = Vh^{[t-1]} + Wx^{[t]}$, no activation. On a 30-word review, how much does token 1 influence the state at token 30?

$$\frac{dh_{30}}{dh_1} = V^{29} = 0.8^{29}$$

Guess the value.

Predict-first: the fading number

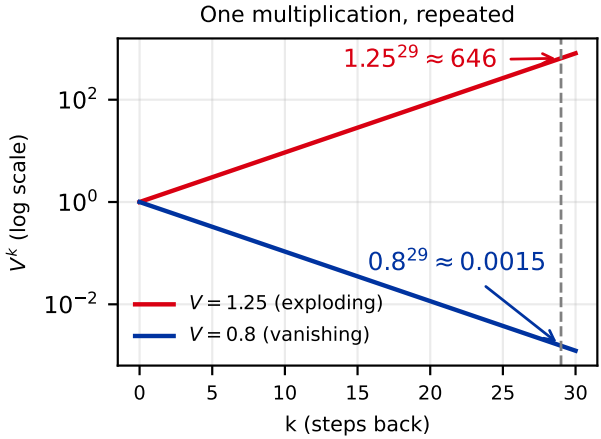
Strip the net down to 1-d: a scalar state $h^{[t]} = Vh^{[t-1]} + Wx^{[t]}$, no activation. On a 30-word review, how much does token 1 influence the state at token 30?

$$\frac{dh_{30}}{dh_1} = V^{29} = 0.8^{29}$$

Guess the value.

$0.8^{29} \approx$ **0.0015**. And if $V = 1.25$? $1.25^{29} \approx$ **646**. One multiplication, repeated, either starves or explodes.

[29]'s smoothing had the same 0.8. There, fading is the point; in language it is the disease.



From one number to a matrix

In a real RNN, $\mathbf{V}$ is a matrix. Take a small one:

$$\mathbf{V} = \begin{bmatrix} 1.0 & -0.2 \\ -0.2 & 1.0 \end{bmatrix}$$

No entry is bigger than 1. **Predict:** multiply the vector $(1, 0)$ by $\mathbf{V}$, 29 times. Does it shrink or grow?

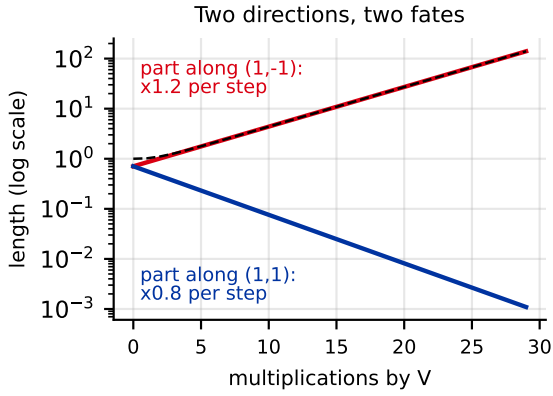
From one number to a matrix

In a real RNN, $\mathbf{V}$ is a matrix. Take a small one:

$$\mathbf{V} = \begin{bmatrix} 1.0 & -0.2 \\ -0.2 & 1.0 \end{bmatrix}$$

No entry is bigger than 1. **Predict:** multiply the vector $(1, 0)$ by $\mathbf{V}$, 29 times. Does it shrink or grow?

It grows, to about **140**$\times$ its length. $\mathbf{V}$ has two special directions, its **eigenvectors**: along $(1, 1)$ it multiplies by $\lambda = 0.8$, along $(1, -1)$ by $\lambda = 1.2$. Split $(1, 0)$ into those two parts and $\mathbf{V}$ only rescales each one: $0.8^{29} \approx 0.0015$, $1.2^{29} \approx 198$.



Dashed: the whole vector. After a few steps it *is* the $\lambda = 1.2$ part: the largest $|\lambda|$ takes over.

The full derivation, part 1

For vectors, the state update is $\mathbf{z}^{[t]} = \sigma(\mathbf{V}^\top \mathbf{z}^{[t-1]} + \mathbf{W}^\top \mathbf{x}^{[t]} + \mathbf{b})$, so by the chain rule:

$$\frac{d\mathbf{z}^{[t]}}{d\mathbf{z}^{[t-1]}} = \operatorname{diag}\big(\sigma'(\cdot)\big)\,\mathbf{V}^\top = \mathbf{D}^{[t-1]}\,\mathbf{V}^\top$$

Chain this back to step 1 - one factor per step:

$$\frac{dL}{d\mathbf{z}^{[1]}} = \frac{dL}{d\mathbf{z}^{[t]}}\,\mathbf{D}^{[t-1]}\mathbf{V}^\top\,\mathbf{D}^{[t-2]}\mathbf{V}^\top \cdots \mathbf{D}^{[1]}\mathbf{V}^\top$$

Every path back to an early time-step multiplies by the **same** matrix $\mathbf{V}^\top$ once per step, $t-1$ times. The $\mathbf{D}$'s sit in between, and they never help: $\tanh' \leq 1$, so each $\mathbf{D}$ can only shrink a vector. If $\mathbf{V}^\top$ alone shrinks every vector, the whole chain shrinks at least as fast - the $\mathbf{D}$'s are a bound, not a rescue.

Why eigenvalues? Along an eigenvector $\mathbf{v}$, the matrix only scales: $\mathbf{V}^\top\mathbf{v} = \lambda\mathbf{v}$, so $(\mathbf{V}^\top)^k\mathbf{v} = \lambda^k\mathbf{v}$ - the scalar story again, one direction at a time. As in the 2×2 example, after many steps the direction with the largest $|\lambda|$ dominates everything else.

The full derivation, part 2 - eigenvalues

The **largest eigenvalue** $|\lambda_{\max}|$ of $\mathbf{V}$ decides the fate of $(\mathbf{V}^\top)^{t-i}$:

- $|\lambda_{\max}| < 1 \Rightarrow$ **vanishing** - and the $\mathbf{D}$ factors (≤ 1 for tanh) can only shrink it further.
- $|\lambda_{\max}| > 1 \Rightarrow$ it **can explode** - unless saturated $\mathbf{D}$ factors happen to cancel the growth.

(Pascanu, Mikolov & Bengio, 2013, make it precise: with tanh, $|\lambda_{\max}| < 1$ is *sufficient* for vanishing, while $|\lambda_{\max}| > 1$ is only *necessary* for exploding.)

Worse than a deep MLP. An MLP multiplies *different* matrices: one layer's shrinking can be undone by the next. An RNN multiplies the **same V** every step, so the same direction shrinks (or grows) every time. Sigmoid $\mathbf{D}$'s make it worse ($|\sigma'| \leq 1/4$).

Callback: [42] already met vanishing gradients in deep MLPs (the δ recursion), and [45] fought them with ReLU and He init. Same disease, worse here - it is the *same* matrix every step.

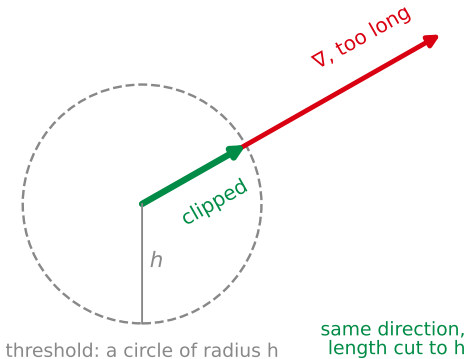
History: this diagnosis is Hochreiter's 1991 diploma thesis (written in German, barely read at the time) and, independently, Bengio, Simard & Frasconi (1994). Remember the name Hochreiter - he returns next lecture with the cure.

Exploding is the easy half

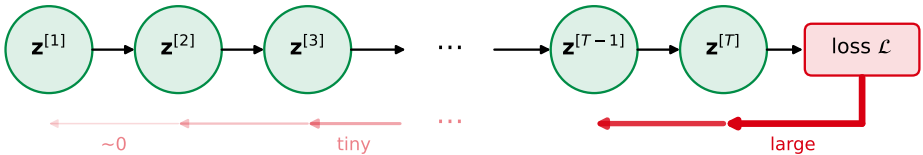
[44] already gave us the fix for cliffs - **gradient clipping**: cap the gradient's norm at a threshold h:

$$\text{if } \|\nabla\| > h: \quad \nabla \leftarrow \frac{h}{\|\nabla\|}\nabla$$

One formula (Pascanu, Mikolov & Bengio, 2013), solved problem. Exploding gradients are the *easy* half of this story.



What vanishing means in practice



shading = how much gradient from "were" reaches each word (illustrative: 0.8 per step)

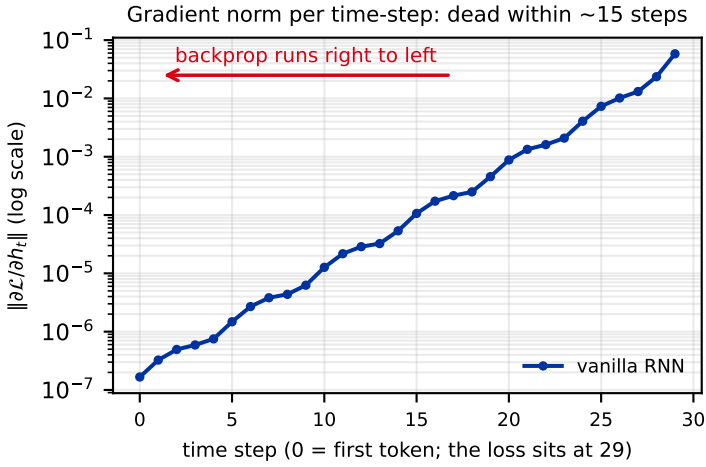
$\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[1]}} = \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[T]}} \cdot \prod_{t=2}^{T} \frac{\partial \mathbf{z}^{[t]}}{\partial \mathbf{z}^{[t-1]}}$: a product of many factors that shrink, so the gradient vanishes and the RNN *forgets* early tokens - here it cannot learn that "were" agrees with "pomegranates".

Our version: the 42-word review from the shoehorn-hacks frame - its verdict, *disappointing*, sits in the last clause. By the time the RNN reaches it, the state has forgotten most of what came before.

Empirical proof

Gradient norm per time-step, measured directly on a tiny vanilla RNN (hidden size 16). The task: read 30 random numbers, then output the **first** one - so the loss at the end needs the gradient to reach step 1.

Read it right to left. The loss sits at step 29. Each step back keeps about **64%** of the gradient (log scale: one grid line = 10×). Step 0, which holds the answer, gets **1/350,000** of it.



Recap

- Text refuses to fit a fixed input: length varies, order matters - every shoehorn hack loses one or the other, and a fixed window is blind beyond k.
- State = a running summary; **one new weight matrix V** turns a dense layer into an RNN. Batches still need a rectangle: **pad, then mask** - pads are not neutral.
- Unrolled = a deep network with **tied weights** - autograd handles it unchanged.
- Same-matrix powers $\Rightarrow$ gradients **vanish** (derived, not asserted) or explode.
- Clipping fixes exploding. **Nothing yet fixes vanishing** - so a vanilla RNN's "unlimited memory" is, in practice, a few dozen steps: no better than a wide fixed window.

Next: 1997's answer - Hochreiter & Schmidhuber's **Long Short-Term Memory** (**LSTM**), a memory built from **addition**, not multiplication. It later runs Google Translate and Siri. And it still will not be enough ([54]).